



**MODULE: B9DA111 DATA STORAGE SOLUTIONS FOR DATA ANALYTICS
(B9DA111_2425_TMD3)**

ASSESSMENT: CA_1

GROUP MEMBER:

Harsh Vardhan (20066051)

Harsanbruno Maria Joseph Esuraj (20065575)

Gunasekhar Vasireddy (20061005)

Proof of Concept: Data Warehouse for Railway Ticketing and Journey Analytics

1. Introduction

Railway companies handle huge amounts of data every day—everything from ticket sales and passenger movements to payment details, train schedules, and interactions with customers. But even though this data is incredibly valuable, much of it does not get used to its full potential. This happens because the information often exists in different formats, is not well connected, or companies simply do not have the right tools to make sense of it all.

This Proof of Concept (PoC) tackles that problem by introducing a dedicated Data Warehouse, built around a dimensional modelling approach. The goal is to take all those scattered bits of raw information and bring them together in one organised, structured place that is designed for efficient analysis. By using a star schema with fact and dimension tables, the warehouse lets teams quickly answer questions, spot trends, and make decisions backed by data.

For this project, we are working with a wide variety of details: transaction IDs, travel dates, departure and arrival stations, ticket types, class of travel, how people paid, whether a railcard was used, prices, refunds, and journey status. All this data is cleaned up, linked together, and arranged in a flexible schema that supports rich reporting and analytics from multiple perspectives.

With this new data warehouse in place, different teams—such as operations, finance, and customer service—can easily dive into historical trends, monitor important performance metrics, track patterns in refunds, and understand how passengers are travelling throughout the network. This platform also lays the groundwork for creating interactive dashboards in tools like SSRS or Tableau, helping railway organisations improve planning, boost revenue, and make smarter strategic choices.

2. Subject Area and Justification

Selected Subject Area: Railway Ticketing and Journey Analytics

Justification:

We chose this area because it delivers significant value to the business and involves a wealth of detailed information. Ticket sales are a primary source of income for railways, while journey records shed light on how reliable and efficient the services are. Analysing refund requests can uncover areas where customers are unhappy or where processes are breaking down. By looking at all these data points together, the organization can fine-tune services, predict revenue more accurately, and create a better experience for travellers.

This focus area also lends itself well to in-depth, multi-angle analysis and supports a robust, scalable data warehouse structure. Plus, it closely supports the objectives of various teams across the company, including operations, finance, marketing, and customer support.

3. Key Stakeholders and Their Objectives

The implementation of a data warehouse directly supports a range of stakeholders across the railway organization, each with unique goals and responsibilities. By centralizing critical operational data, the

warehouse ensures that every team has access to consistent, accurate, and timely information to drive smarter decision-making.

- **Operations Management** relies on the warehouse to track train performance, identify routes with frequent delays, and adjust scheduling to improve service reliability and efficiency.
- **Finance Teams** use the data to evaluate revenue trends, measure the financial effects of ticket refunds, and assess how discounts and promotions are impacting profitability.
- **Sales and Marketing Departments** gain insights into customer purchasing patterns, helping them understand which products or ticket types are most popular—and where opportunities exist to promote higher-margin offerings.
- **Customer Support Teams** benefit from access to journey and refund data, allowing them to respond more effectively to complaints, monitor service issues, and take proactive steps to enhance the customer experience.
- **IT and Data Teams** are responsible for maintaining the backend infrastructure, ensuring that ETL processes are running smoothly, data quality is upheld, and access to analytics tools is secure and reliable.

Across all departments, the data warehouse acts as a single source of truth—eliminating data silos, improving reporting accuracy, and enabling cross-functional collaboration.

4. Business Vision

Building a data warehouse is a strategic step toward helping the organization make decisions based on solid evidence. By converting raw transactional data into a format that's ready for analysis, this warehouse empowers different departments to review past results, spot inefficiencies, and plan smarter improvements for the future.

The main goals include:

- Creating a centralized system that brings together data from various sources.
- Enabling trend tracking, historical reporting, and gaining operational insights.
- Ensuring smooth integration with business intelligence tools for easy dashboard creation and reporting.
- Offering business users, the ability to perform their own analyses without heavy IT support.
- Setting the stage for advanced capabilities like forecasting and machine learning applications.

Ultimately, this data warehouse acts as the organization's key repository for monitoring essential performance metrics, evaluating business outcomes, and pinpointing opportunities for operational or financial enhancements.

5. Dimensional Model Design

To enable fast, user-friendly analytics, the data warehouse is structured using a **star schema**. This architecture places a central **fact table** at its core, surrounded by several connected **dimension tables**

that provide the descriptive details needed for meaningful analysis. The simplicity and performance of this model make it ideal for slicing, dicing, and aggregating data across multiple perspectives.

5.1 Fact Table: Journey Transactions

At the centre of the schema lies the **Journey Transactions Fact Table**, which houses the measurable events linked to each customer interaction with the railway system. This includes data related to ticket purchases, journey timings, and service outcomes.

Key Metrics Captured:

- **Ticket Price:** The revenue earned from each ticket transaction.
- **Departure & Arrival Times:** Scheduled travel times for the journey.
- **Actual Arrival Time:** Recorded arrival time, used to measure delays.
- **Journey Status:** Indicates whether the train was on time or delayed.
- **Refund Request:** Flags whether a customer requested a refund for the journey.

Foreign Keys and Relationships:

- **Purchase Date & Journey Date:** Connected to the **Date Dimension** for time-based insights.
- **Departure & Arrival Stations:** Linked to the **Station Dimension** to allow for geographic and route-specific analysis.
- **Ticket Type:** Related to the **Ticket Dimension** to evaluate pricing strategies and customer preferences.
- **Payment Method & Purchase Channel:** Mapped to the **Payment Dimension** to analyse buying behaviour and payment trends.

The fact table forms the foundation for key reporting functions—such as tracking punctuality, calculating revenue, analysing refund patterns, and evaluating how different ticket types and discounts influence performance.

5.2 Dimension Tables: Descriptive Context

Dimension tables support the fact table by offering the **descriptive attributes** necessary for breaking down and interpreting raw metrics. Each dimension contains a surrogate key and fields that describe one of the main entities involved in a transaction.

Date Dimension

Stores key temporal data, enabling powerful time-based filtering and trend discovery.

- **Attributes:**
 - Full Date
 - Day, Month, Year
 - Day of the Week

- **Business Purpose:**
Enables analysis of sales and traffic patterns across time. For example, stakeholders can compare weekday vs. weekend travel, identify seasonal demand peaks, and track revenue growth over different periods.

Station Dimension

Captures details about departure and arrival stations along with their geographical metadata.

- **Attributes:**
 - Station Code
 - Station Name
 - Region
 - Zone
- **Business Purpose:**
Helps analyse performance at a regional level—such as identifying the busiest stations, monitoring high-revenue routes, and highlighting zones prone to delays or disruptions.

Ticket Dimension

Describes the characteristics of tickets sold, helping assess customer choices and product strategies.

- **Attributes:**
 - Ticket Class (e.g., Standard, First Class)
 - Ticket Type (e.g., Advance, Anytime)
 - Railcard Type (e.g., Student, Senior)
 - Discount Percentage
- **Business Purpose:**
Enables financial analysis of fare structures, tracks usage of railcards and discounts, and helps optimize ticketing offerings based on customer behaviour and revenue impact.

Payment Dimension

Provides information about how transactions were completed, combining purchase methods and sales channels.

- **Attributes:**
 - Purchase Type (e.g., Online, At Station)
 - Payment Method (e.g., Credit Card, Contactless)

Business Purpose:

Supports marketing and IT teams in assessing consumer preferences and monitoring the adoption of digital channels. It is particularly useful for evaluating the success of contactless payments and online ticketing campaigns.

Strategic Benefits

By organizing railway transaction data into this star schema, the warehouse empowers teams to perform advanced analytics efficiently. It transforms raw data into a reliable foundation for tracking operational KPIs, understanding customer behaviour, and driving strategic improvements across departments.

6. Business Insights Enabled by the Data Warehouse

The implementation of a structured data warehouse allows the railway organization to transform raw, disconnected data into meaningful insights. These insights provide the foundation for strategic decisions that drive operational efficiency, customer satisfaction, and revenue growth. By aligning the data warehouse with key performance indicators across departments, the organization can unlock a variety of analytical capabilities.

Operational Performance Monitoring

With integrated data on journey timings, delays, and routes, operations teams can closely monitor service delivery and identify performance gaps.

- Track on-time arrival rates by individual stations and entire routes.
- Detect recurring delays on specific lines and correlate them with customer refund requests.
- Identify time windows—such as peak hours or specific days—when delays are more frequent than average.

Revenue and Sales Analysis

By consolidating ticket sales, pricing models, and discount programs, the finance team can perform detailed financial analysis at multiple levels.

- Break down total revenue by station, ticket type, or date to identify high-earning segments.
- Measure how different railcards and promotional discounts affect overall revenue.
- Compare sales trends over time to assess growth or decline in demand by season or year.

Customer Behaviour Insights

The warehouse captures data on purchasing methods and ticket preferences, enabling marketing and planning teams to understand customer habits.

- Analyse whether passengers prefer buying tickets online, at stations, or through other channels.
- Identify the most popular ticket types (e.g., Advance, Anytime) and classes (e.g., First Class, Standard).
- Evaluate the usage of various payment methods to guide future upgrades and initiatives.

Refund Trend Analysis

Refund data, when examined alongside journey performance, reveals valuable insights into service quality and customer pain points.

- Monitor patterns in refund requests based on route, station, or ticket category.
- Quantify the financial implications of frequent delays that lead to refunds.

- Use refund frequency to highlight service reliability concerns or customer dissatisfaction trends.

Strategic Value

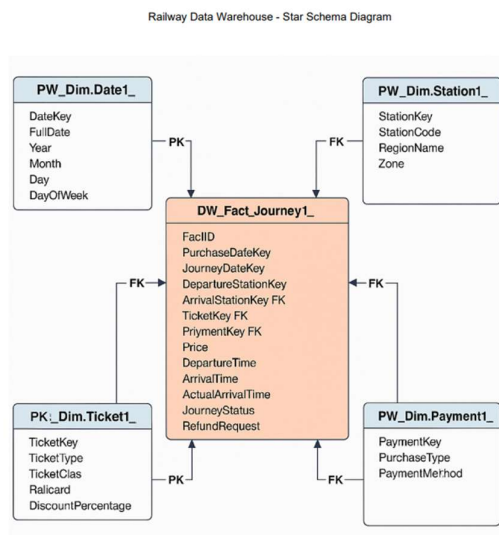
These insights empower various departments to move from reactive to proactive planning. Whether it is adjusting train schedules, redesigning ticket offers, or investing in infrastructure, the data warehouse provides the factual grounding needed to make smarter, evidence-based decisions across the organization.

7. Strategic Benefits and Future Scope

Strategic Benefits

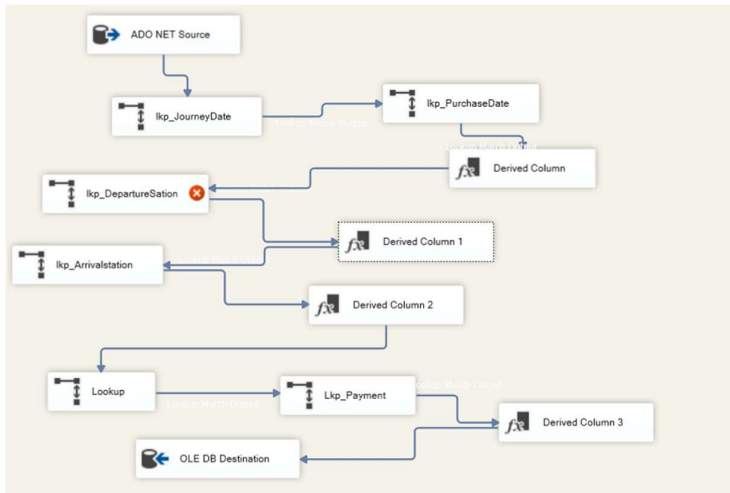
- Establishes a centralized, consistent source of truth for all key performance metrics.
- Reduces reliance on manual data extraction and fragmented reports.
- Promotes a culture of data-driven decision-making across departments.
- Supports automation of regular reporting and KPI dashboards.

2. Schema Design and Justification



The schema used in the railway data warehouse operates under star schema architecture that makes it perfect in analytical processing since it is simple, scalable, and efficient. At the core of the model is the fact table **DW_Fact_Journey1_**, which comprises of all the measurable data that is journey specific such as journey and purchase dates, price, and travel times, journey status and refund details. This table is related to the following four-dimension tables: **DW_Date1_**, **DW_Station1_**, **DW_Ticket1_**, and **DW_Payment1_**. The dimension tables can hold descriptive attributes of dates, stations, ticket details, and payment methods respectively, so that the users can slice the journey data in various perspectives and analyse the data. The reason as to why this design was adopted is to enable quick and easy reporting on the performance indicators like top stations, journey delay, ticket class preference and payment trends. It enables effective querying of the business intelligence and data integrity and modularity and as such is the best option in creating an insightful and maintainable data warehouse.

3. ETL



ETL Process for Populating the Railway Data Warehouse Using SSIS

To efficiently transfer railway transaction data from external sources into the data warehouse, a structured ETL (Extract, Transform, Load) process was designed and implemented using SQL Server Integration Services (SSIS). The pipeline follows a modular and performance-optimized approach, incorporating multiple stages of validation and transformation to ensure data quality and integrity.

Overview of the SSIS Workflow

The ETL workflow comprises several components organized within an SSIS Data Flow Task. This pipeline facilitates the seamless flow of data—from raw input to final warehouse storage—while applying key transformation and lookup logic along the way.

ETL Stages

1. Data Extraction (Source)

- The process initiates with an ADO.NET Source component, which connects to an external data source—typically an Excel file or a staging database containing raw railway transactions.
- **Fields extracted include:**
 - Purchase Date and Journey Date
 - Departure and Arrival Station Names
 - Ticket Type and Payment Method
 - Ticket Price, Journey Status, and Refund Status

This step ensures that all necessary attributes required for analysis and warehouse loading are captured upfront.

2. Lookup Transformations (Dimension Mapping)

To link incoming data with the appropriate dimension records in the warehouse, several Lookup components are employed:

- **lkp_JourneyDate and lkp_PurchaseDate**
Match date fields from the source with entries in the **dw_date1_dimension**, retrieving surrogate keys for both journey and purchase dates.
- **lkp_DepartureStation and lkp_ArrivalStation**
Identify and map station names to the **dw_station1_** dimension, returning keys for both the departure and destination points.
- **lkp_Payment**
Connects payment methods and purchase channels from the source with entries in the **dw_payment1_** dimension table.

Each lookup operation enforces referential integrity by replacing business (natural) keys with surrogate keys used by the fact table.

3. Derived Column Transformations

To standardize and clean the extracted data, a Derived Column transformation is introduced:

- It modifies or reformats key fields such as station names, ticket types, and payment methods into a consistent format.
- This ensures that all incoming data adheres to naming conventions and quality standards before loading into the data warehouse.

4. Data Loading (Destination)

- Once transformed, the data flows into an OLE DB Destination, which writes the finalized records into the fact table within the data warehouse schema.
- The destination table maintains proper foreign key relationships by referencing surrogate keys from the associated dimension tables.

This step completes the ETL cycle, ensuring clean, consistent, and relationally accurate data is stored in the warehouse.

Key Features of the ETL Pipeline

Feature	Description
Parallel Lookup Execution	Lookup components are configured to run in parallel, improving overall speed.
Data Standardization	Derived columns ensure that key text fields follow consistent formatting rules.
Referential Integrity	Surrogate keys are retrieved via lookup to maintain accurate table relationships.

Data Validation and Verification

After executing the SSIS ETL package, data integrity was verified using SQL queries within SQL Server Management Studio (SSMS). Each table was checked for completeness and correctness:

- The **dw_date1_** table contains properly formatted calendar dates, with details such as year, month, day, and weekday.
- The **dw_station1_** table holds unique station codes and names.
- The **dw_ticket1_** table includes various ticket types, travel classes, and applicable railcard categories.
- The **dw_payment1_** table reflects different combinations of purchase methods and transaction types.
- The **dw_fact_journey1_** fact table successfully references all relevant dimension tables using surrogate keys and captures key transactional details including price, timing, journey status, and refund requests.

This thorough validation confirms that the ETL process accurately extracted, transformed, and loaded data into the warehouse according to the dimensional model, supporting high-quality analytics downstream.

```
select * from DW Dim Payment1
```

	PaymentKey	PurchaseType	PaymentMethod
1	1	Online	Debit Card
2	2	Station	Debit Card
3	3	Online	Credit Card
4	4	Station	Credit Card
5	5	Station	Contactless
6	6	Online	Contactless

```
select * from DW Dim Station1
```

	StationKey	StationCode	StationName	Region	Zone
1	1	Birmingham New Street	Birmingham New Street	NULL	NULL
2	2	Bristol Temple Meads	Bristol Temple Meads	NULL	NULL
3	3	Cardiff Central	Cardiff Central	NULL	NULL
4	4	Coventry	Coventry	NULL	NULL
5	5	Crewe	Crewe	NULL	NULL
6	6	Didcot	Didcot	NULL	NULL
7	7	Doncaster	Doncaster	NULL	NULL
8	8	Durham	Durham	NULL	NULL
9	9	Edinburgh	Edinburgh	NULL	NULL
10	10	Edinburgh Waverley	Edinburgh Waverley	NULL	NULL
11	11	Leeds	Leeds	NULL	NULL
12	12	Leicester	Leicester	NULL	NULL
13	13	Liverpool Lime Street	Liverpool Lime Street	NULL	NULL
14	14	London Euston	London Euston	NULL	NULL
15	15	London Kings Cross	London Kings Cross	NULL	NULL
16	16	London Paddington	London Paddington	NULL	NULL

```
select * from DW Dim Ticket1
```

	TicketKey	TicketType	TicketClass	Railcard	DiscountPercentage
1	1	Off-Peak	First Class	Senior	NULL
2	2	Anytime	Standard	Disabled	NULL
3	3	Anytime	Standard	Senior	NULL
4	4	Anytime	First Class	None	NULL
5	5	Advance	Standard	Adult	NULL
6	6	Advance	First Class	Senior	NULL
7	7	Advance	First Class	Disabled	NULL
8	8	Off-Peak	First Class	Disabled	NULL
9	9	Anytime	First Class	Adult	NULL
10	10	Advance	Standard	None	NULL
11	11	Off-Peak	First Class	Adult	NULL
12	12	Anytime	Standard	Adult	NULL
13	13	Off-Peak	Standard	Senior	NULL
14	14	Off-Peak	Standard	Disabled	NULL
15	15	Off-Peak	Standard	Adult	NULL
16	16	Advance	First Class	None	NULL
17	17	Advance	Standard	Senior	NULL

```
select * from DW Dim Date1
```

	DateKey	FullDate	Year	Month	Day	DayOfWeek
1	1	2023-12-19 00:00:00.000	2023	12	19	Tuesday
2	2	2023-12-31 00:00:00.000	2023	12	31	Sunday
3	3	2023-12-20 00:00:00.000	2023	12	20	Wednesday
4	4	2023-12-16 00:00:00.000	2023	12	16	Saturday
5	5	2023-12-27 00:00:00.000	2023	12	27	Wednesday
6	6	2023-12-08 00:00:00.000	2023	12	8	Friday
7	7	2023-12-30 00:00:00.000	2023	12	30	Saturday
8	8	2024-01-09 00:00:00.000	2024	1	9	Tuesday
9	9	2024-04-11 00:00:00.000	2024	4	11	Thursday
10	10	2024-04-13 00:00:00.000	2024	4	13	Saturday
11	11	2024-02-06 00:00:00.000	2024	2	6	Tuesday
12	12	2024-03-20 00:00:00.000	2024	3	20	Wednesday
13	13	2024-03-08 00:00:00.000	2024	3	8	Friday
14	14	2024-01-10 00:00:00.000	2024	1	10	Wednesday
15	15	2024-03-31 00:00:00.000	2024	3	31	Sunday
16	16	2024-01-14 00:00:00.000	2024	1	14	Sunday

SELECT * FROM Dw_Fact_Journey1													
110 %													
Results Messages													
	FactID	PurchaseDateKey	JourneyDateKey	DepartureStationKey	ArrivalStationKey	TicketKey	PaymentKey	Price	DepartureTime	ArrivalTime	ActualArrivalTime	JourneyStatus	RefundRequest
1	1	70	110	13	19	5	4	2	21:30:00.0000000	22:00:00.0000000	22:00:00.0000000	On Time	No
2	2	70	110	16	24	10	4	13	22:15:00.0000000	23:15:00.0000000	23:15:00.0000000	On Time	No
3	3	70	110	14	1	5	4	4	22:15:00.0000000	23:35:00.0000000	23:35:00.0000000	On Time	No
4	4	110	110	19	13	24	3	4	01:30:00.0000000	02:00:00.0000000	02:44:00.0000000	Delayed	Yes
5	5	110	110	14	22	15	4	3	22:30:00.0000000	23:40:00.0000000	23:40:00.0000000	On Time	No
6	6	110	110	19	14	15	4	84	01:45:00.0000000	03:35:00.0000000	03:35:00.0000000	On Time	No
7	7	110	110	15	13	24	6	97	01:45:00.0000000	04:00:00.0000000	04:00:00.0000000	On Time	No
8	8	110	54	15	10	22	6	24	16:00:00.0000000	20:20:00.0000000	20:20:00.0000000	On Time	No
9	9	110	110	19	20	24	3	17	02:00:00.0000000	03:00:00.0000000	03:00:00.0000000	On Time	No
10	10	110	54	15	10	7	6	48	16:00:00.0000000	20:20:00.0000000	20:20:00.0000000	On Time	No
11	11	110	110	13	19	24	3	5	02:15:00.0000000	02:45:00.0000000	02:45:00.0000000	On Time	No
12	12	110	84	13	19	10	3	3	23:30:00.0000000	00:00:00.0000000	00:00:00.0000000	On Time	No
13	13	110	84	16	24	10	4	13	23:30:00.0000000	00:30:00.0000000	00:30:00.0000000	On Time	No

4. SSRS Reports Overview

To transform raw data into actionable insights, a suite of interactive and visually structured reports was developed using SQL Server Reporting Services (SSRS). Each report caters to different stakeholder needs, enabling dynamic data exploration, performance tracking, and decision-making.

1. Tabular Report

Title: Top 50 Train Ticket Prices – Detailed Journey Information

Report Type: Tabular Report

This report presents detailed, transaction-level records, focusing on the top 50 tickets by price. It provides a comprehensive view of each journey, including:

- Unique transaction identifiers and key travel dates
- Departure and arrival station names
- Ticket type, travel class, payment method, journey status, and ticket price

Use Case:

Designed to support pricing analysis and route performance, this report helps stakeholders evaluate which routes generate the highest revenue and understand ticket pricing trends.

Top 50 Train Ticket Prices – Detailed Journey Information

Fact ID	Purchase Date	Journey Date	Departure Station	Arrival Station	Ticket Type
7434	1/9/2024 12:00:00 AM	1/9/2024 12:00:00 AM	Manchester Piccadilly	London Euston	Anytime
10707	1/4/2024 12:00:00 AM	1/4/2024 12:00:00 AM	Manchester Piccadilly	London Euston	Anytime
18944	2/9/2024 12:00:00 AM	2/9/2024 12:00:00 AM	Manchester Piccadilly	London Euston	Anytime
26661	2/9/2024 12:00:00 AM	2/9/2024 12:00:00 AM	Manchester Piccadilly	London Euston	Anytime
32580	1/4/2024 12:00:00 AM	1/4/2024 12:00:00 AM	Manchester Piccadilly	London Euston	Anytime
34800	1/9/2024 12:00:00 AM	1/9/2024 12:00:00 AM	Manchester Piccadilly	London Euston	Anytime

2. Matrix Report

Title: Revenue by Ticket Class and Payment Method

Report Type: Matrix Report

This report summarizes revenue performance by ticket class (e.g., Standard, First Class) and payment method (e.g., Credit Card, Contactless), using a matrix layout for cross-tabulated analysis.

Use Case:

Ideal for sales and marketing teams, it helps uncover customer preferences, informs pricing strategies, and evaluates the impact of specific payment methods on total revenue.

Revenue by Ticket Class and Payment Method (MATRIX REPORT)

	Contactless
First Class	93453
Standard	360498

3. Parameterised Report

Title: Ticket Sales Summary by Departure Station and Journey Date

Report Type: Parameterised Report

This dynamic report allows users to input specific journey dates and filter results accordingly. The report then displays ticket sales metrics segmented by departure station.

Use Case:

Enables operational managers to track travel demand across stations on a day-by-day basis. This supports short-term planning, resource allocation, and response to changing passenger flow patterns.

Ticket Sales Summary by Departure Station and Journey Date (PARAMETRISED REPORT)		1/2/2024 12:00:00 AM	
1/1/2024 12:00:00 AM		Birmingham New Street	42
Birmingham New Street		Bristol Temple Meads	
Bristol Temple Meads		Edinburgh Waverley	
Edinburgh Waverley		Liverpool Lime Street	750
Liverpool Lime Street		London Euston	105
London Euston		London Kings Cross	429
London Kings Cross		London Paddington	123
		London St Pancras	807
		Manchester Piccadilly	1590

4. Drill-up/Drill-down Report

Title: Zone-wise Station Revenue Report

Report Type: Drill-up/Drill-down Report

Designed for executive-level insights, this report offers the ability to drill from high-level (zone-wide) revenue summaries down into specific stations within each zone.

Use Case:

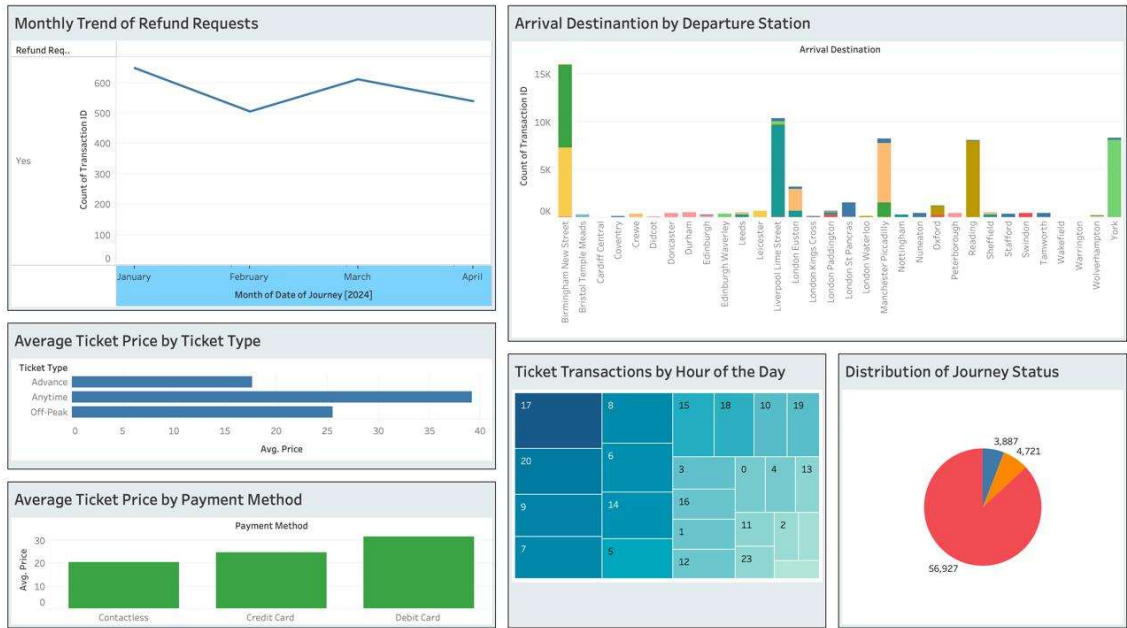
Ideal for strategic planning, this report helps regional managers and executives identify top-performing locations, evaluate zone-level profitability, and inform investment or staffing decisions.

**Zone-wise Station Revenue Report(DRILL-UP/
DRILL-DOWN REPORT)**

Birmingham New Street	78831
Bristol Temple Meads	196
Edinburgh Waverley	4272
Liverpool Lime Street	280267
London Euston	231797
London Kings Cross	414838
London Paddington	173745
London St Pancras	129884
Manchester Piccadilly	156807
Oxford	5842
Reading	21084
York	40750

4. Tableau Visualisations and Dashboard

Further to help sustain business intelligence considering the developed data warehouse, four insightful visualisations that incorporated the developed Tableau Dashboard were generated. The visuals have a direct connection to the subject area chosen (data about railway journeys) and help the key stakeholders to achieve their goals to obtain operational and strategic data.



1. Monthly Refund Request Trend

Insight: A line chart visualizes the monthly count of refund requests marked as "Yes" from January to April 2024.

Business Value:

- Enables customer service and operations teams to detect periods of increased dissatisfaction.
- Helps management correlate spikes in refunds with delays, cancellations, or seasonal variations.
- Encourages proactive policy changes during months with a higher volume of refund requests.

2. Popular Routes: Arrival by Departure

Insight: A bar chart shows the volume of journeys (based on Transaction ID) from each departure station to various arrival destinations.

Business Value:

- Identifies high-traffic routes, helping with schedule optimization and resource allocation.
- Assists station managers in planning for capacity needs.
- Useful for forecasting demand and making data-backed decisions about train frequency.

3. Average Ticket Price Breakdown

Two bar charts analyse average ticket prices from different angles:

a) By Ticket Type

- Compares prices across Advance, Anytime, and Off-Peak tickets.
- Insight: Anytime tickets are the most expensive, aligning with their premium flexibility.

b) By Payment Method

- Displays average spend by Contactless, Credit Card, and Debit Card users.
- Insight: Debit Card users tend to spend more, possibly suggesting a preference for flexible ticket options.

Business Value:

- Helps pricing teams adjust or promote ticket types for better revenue optimization.
- Supports marketing teams in crafting targeted promotions based on payment behaviour.

4. Journey Status Distribution

Insight: A pie chart shows the proportion of journeys categorized as On Time, Delayed, or Cancelled.

Business Value:

- Provides a snapshot of operational efficiency.
- Helps quality assurance teams evaluate service reliability.
- Informs transport authorities about areas needing improvement or investment.

5. Ticket Sales by Hour

Insight: A heatmap displays ticket transaction volumes across each hour of the day.

Business Value:

- Reveals peak purchase periods, guiding staff scheduling and system performance planning.
- Assists with workforce management and helps ensure customer support readiness.
- Enables off-peak promotional strategies to increase sales during low-traffic hours.

Dashboard Relevance for Stakeholders

This dashboard delivers critical metrics and trends valuable for:

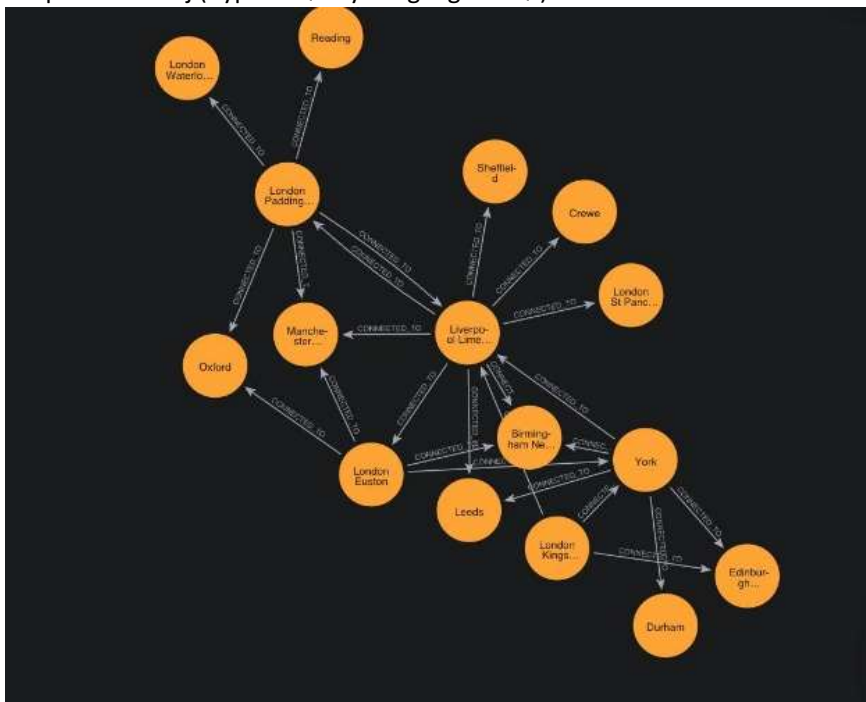
- Operations Managers – to optimize train routes and resource distribution
- Revenue and Ticketing Teams – to improve pricing models and payment analysis
- Marketing Analysts – for data-driven customer engagement strategies
- Customer Service and Quality Assurance Teams – to monitor service performance and enhance the passenger experience

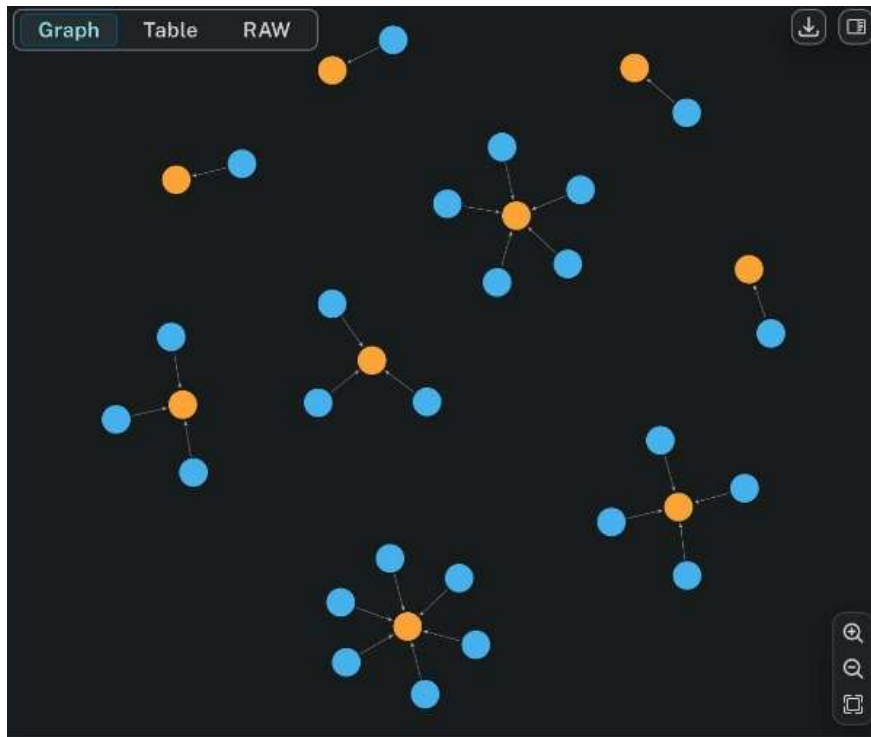
5. Relational vs Graph Database Query Comparison

Dataset: Railway Journey Transactions

Relational DB: SQL Server

Graph DB: Neo4j (Cypher Query Language - CQL)





1. Journey Count per Departure Station

SQL Query:

```
SELECT [Departure Station], COUNT(*) AS JourneyCount
FROM Sheet1$
GROUP BY [Departure Station]
ORDER BY JourneyCount DESC;
```

This query calculates how many journeys start from each departure station. It groups the records by [Departure Station], counts the number of entries in each group, and sorts them from highest to lowest.

----query 1 Top Departure Stations by Journey Count---

```
SELECT
  [Departure Station],
  COUNT(*) AS JourneyCount
FROM
  Sheet1$
GROUP BY
  [Departure Station]
ORDER BY
  JourneyCount DESC;
```

Departure Station	JourneyCount
Manchester Piccadilly	11608
London Euston	10240
Liverpool Lime Street	9461
London Paddington	9321
London Kings Cross	8770
London St Pancras	8048
Birmingham New St.	4404
York	1933
Reading	1242
Oxford	294
Edinburgh Waverley	104
Bristol Temple Meads	32

Cypher Query:

```
MATCH (s:Station)-[:STARTS_AT]-(j:Journey)
RETURN s.name AS DepartureStation, COUNT(j) AS JourneyCount
ORDER BY JourneyCount DESC;
```


In Neo4j, this query finds all Journey nodes that have a STARTS_AT relationship with a Station node. It counts how many journeys begin at each station and orders the result by the journey count in descending order.

```
1 MATCH (s:Station)-[:STARTS_AT]-(j:Journey)
2 RETURN s.name AS DepartureStation,
3        COUNT(j) AS JourneyCount
4 ORDER BY JourneyCount DESC;
```

TableRAW

	DepartureStation	JourneyCount
1	"Manchester Piccadilly"	5335
2	"London Euston"	4698
3	"Liverpool Lime Street"	4327
4	"London Paddington"	4143
5	"London Kings Cross"	4021
6	"London St Pancras"	3601
7	"Birmingham New Street"	2008

2. Journeys with Price Greater than 50

SQL Query:

SELECT [Transaction ID], [Departure Station], [Arrival Destination], [Price]
FROM Sheet1\$
WHERE [Price] > 50
ORDER BY [Price] DESC;

This retrieves all journeys where the ticket price exceeds 50. It displays details like transaction ID, departure and arrival stations, and the price, ordered by the highest fare.

```
-----query 2 Journeys with Price > 50-----
SELECT
  [Transaction ID],
  [Departure Station],
  [Arrival Destination],
  [Price]
FROM
  Sheet1$
WHERE
  [Price] > 50
ORDER BY
  [Price] DESC;
```

110 %

ResultsMessages

	Transaction ID	Departure Station	Arrival Destination	Price
1	0519347-2107-4b48-8adb	Manchester Piccadilly	London Euston	267
2	092e5598-08de-42f5-b6b2	Manchester Piccadilly	London Euston	267
3	de723602-3979-4d69-9664	Manchester Piccadilly	London Euston	267
4	de723602-3979-4d69-9664	Manchester Piccadilly	London Euston	267
5	092e5598-08de-42f5-b6b2	Manchester Piccadilly	London Euston	267
6	0519347-2107-4b48-8adb	Manchester Piccadilly	London Euston	267
7	0519347-2107-4b48-8adb	Manchester Piccadilly	London Euston	267
8	092e5598-08de-42f5-b6b2	Manchester Piccadilly	London Euston	267
9	d327e0ec-e1ac-438a-a05c	Reading	Liverpool Lime Street	242
10	d327e0ec-e1ac-438a-a05c	Reading	Liverpool Lime Street	242
11	a4bbac34-6ed7-4d71-b738	Reading	Liverpool Lime Street	242
12	a4bbac34-6ed7-4d71-b738	Reading	Liverpool Lime Street	242
13	419bcd59-8bc0-48f5-b48c	Liverpool Lime Street	London Paddington	238
14	8e02cccf-5958-48ca-bbb8	Liverpool Lime Street	London Paddington	238

Cypher Query:

MATCH (j:Journey)-[:STARTS_AT]->(src:Station)-[:CONNECTED_TO]->(dest:Station)
 WHERE j.price > 50
 RETURN j.id AS TransactionID, src.name AS DepartureStation, dest.name AS ArrivalStation, j.price
 ORDER BY j.price DESC;

In the graph database, it matches Journey nodes that are connected from a src (departure station) to a dest (arrival station). It filters those journeys with a price above 50 and lists them by highest price.

```

1 MATCH (j:Journey)-[:STARTS_AT]->(src:Station)-[:CONNECTED_TO]->(dest:Station)
2 WHERE j.price > 50
3 RETURN
4   j.id AS TransactionID,
5   src.name AS DepartureStation,
6   dest.name AS ArrivalStation,
7   j.price
8 ORDER BY j.price DESC;

```

TransactionID	DepartureStation	ArrivalStation	j.price
"092e5598-08de-42f5-b6b2"	"Manchester Piccadilly"	"London Paddington"	267
"05193f47-2107-4bb8-8adb"	"Manchester Piccadilly"	"London Paddington"	267
"de723682-3979-4d69-9664"	"Manchester Piccadilly"	"London Paddington"	267
"092e5598-08de-42f5-b6b2"	"Manchester Piccadilly"	"London Kings Cross"	267
"05193f47-2107-4bb8-8adb"	"Manchester Piccadilly"	"London Kings Cross"	267
"de723682-3979-4d69-9664"	"Manchester Piccadilly"	"London Kings Cross"	267
"092e5598-08de-42f5-b6b2"	"Manchester Piccadilly"	"Liverpool Lime Street"	267

Fetch limit hit at 5000

3. First Class Tickets

SQL Query:

SELECT [Transaction ID], [Departure Station], [Arrival Destination], [Ticket Class]
 FROM Sheet1\$
 WHERE [Ticket Class] = 'First Class';

This selects journeys where the ticket class is 'First Class'. It shows relevant details like transaction ID, departure, and arrival locations.

```

-- SELECT
--   [Transaction ID],
--   [Departure Station],
--   [Arrival Destination],
--   [Ticket Class]
-- FROM
--   Sheet1$
-- WHERE
--   [Ticket Class] = 'First Class';

```

-- query 4 Stations with More Than 5 Departing Journeys----

Transaction ID	Departure Station	Arrival Destination	Ticket Class
0b367247d01-4a03-9e60	London Kings Cross	Edinburgh Waverley	First Class
65e3e780-e004-4899-8706	Manchester Piccadilly	London Paddington	First Class
57a7c85b-7db5-485a-8626	London Paddington	Reading	First Class
e4496429-4452-4bbe-89a9	Liverpool Lime Street	Manchester Piccadilly	First Class
d26a8544-fac1-4595-b699	Manchester Piccadilly	Liverpool Lime Street	First Class
b65e17ab-c0a3-40f1-ea40	Birmingham New Street	London Kings Cross	First Class
a94d434b-3925-4eab-8b27	Liverpool Lime Street	London Euston	First Class
5fab6974-305d-4728-826c	London Euston	Manchester Piccadilly	First Class
10274172-62d2-445e-a048	London Paddington	Reading	First Class
9ba12b59-6e74-48a0-83c3	London Kings Cross	York	First Class
522e86c1-2ba6-4c37-8d7e	London Paddington	Reading	First Class
5e6b8815-6e54-4e85-87d5	Liverpool Lime Street	London Euston	First Class
a34c397d167a-413c-87d8	Liverpool Lime Street	London Euston	First Class
c32c29ad-8bc7-47d5-bb3d	Birmingham New Street	Manchester Piccadilly	First Class
db843955-7d84-4081-bd18	London Euston	Birmingham New Street	First Class

Cypher Query:

MATCH (j:Journey)-[:STARTS_AT]->(src:Station)-[:CONNECTED_TO]->(dest:Station)
 WHERE j.ticket_class = 'First Class'

RETURN j.id AS TransactionID, src.name AS DepartureStation, dest.name AS ArrivalStation,
j.ticket_class;

This graph query locates journeys labelled with 'First Class' tickets and retrieves associated station information, similar to the SQL output.

```
1 MATCH (j:Journey)-[:STARTS_AT]->(src)-[:CONNECTED_TO]->(dest)
2 WHERE j.ticket_class = 'First Class'
3 RETURN
4   j.id AS TransactionID,
5   src.name AS DepartureStation,
6   dest.name AS ArrivalStation,
7   j.ticket_class;
```

TransactionID	DepartureStation	ArrivalStation	j.ticket_class
"148cc904-8fa9-4e28-ac1e"	"London Paddington"	"Liverpool Lime Street"	"First Class"
"d8f26c88-4e2e-4cca-af4b"	"London Paddington"	"Liverpool Lime Street"	"First Class"
"08043a64-07cf-4f17-bf18"	"London Paddington"	"Liverpool Lime Street"	"First Class"
"bb28209f-3487-460e-a099"	"London Paddington"	"Liverpool Lime Street"	"First Class"
"410b723e-da3b-473b-8a66"	"London Paddington"	"Liverpool Lime Street"	"First Class"
"96f4660d-6f44-4a37-81b3"	"London Paddington"	"Liverpool Lime Street"	"First Class"
"9e5cd84c-1f56-4a20-0a16"	"London Paddington"	"Liverpool Lime Street"	"First Class"

Fetch limit hit at 5000 records.

4. Stations with More Than 5 Departing Journeys

SQL Query:

```
SELECT [Departure Station], COUNT(*) AS JourneysDeparted
FROM Sheet1$
GROUP BY [Departure Station]
HAVING COUNT(*) > 5
ORDER BY JourneysDeparted DESC;
```

The query counts how many journeys start at each station and filters for stations that have more than 5 departures. It then displays those stations along with the count of departing journeys.

```
-- query 4 Stations with More Than 5 Departing Journeys---
SELECT
  [Departure Station],
  COUNT(*) AS JourneysDeparted
FROM
  Sheet1$
GROUP BY
  [Departure Station]
HAVING
  COUNT(*) > 5
ORDER BY
  JourneysDeparted DESC;
```

Departure Station	JourneysDeparted
Manchester Piccadilly	11606
London Euston	10240
Liverpool Lime Street	9461
London Paddington	9321
London Kings Cross	8770
London St Pancras	8048
Birmingham New Street	4404
York	1933
Reading	1242
Oxford	294
Edinburgh Waverley	104
Bristol Temple Meads	32

Cypher Query:

```
MATCH (s:Station)-[:STARTS_AT]->(j:Journey)
WITH s, COUNT(j) AS JourneysDeparted
WHERE JourneysDeparted > 5
RETURN s.name AS DepartureStation, JourneysDeparted
ORDER BY JourneysDeparted DESC;
```

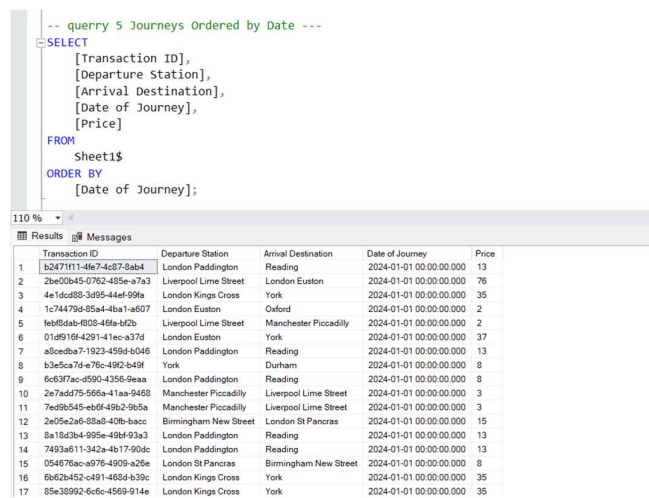
This matches stations linked with journeys through STARTS_AT, counts how many journeys are linked per station, and filters those with more than five. The result is sorted by the number of departures.

5. Journeys Ordered by Date

SQL Query:

```
SELECT [Transaction ID], [Departure Station], [Arrival Destination], [Date of Journey], [Price]
FROM Sheet1$
ORDER BY [Date of Journey];
```

This simply displays journey records ordered by their journey date, allowing for chronological tracking of trips.



```
-- query 5 Journeys Ordered by Date ---
SELECT
    [Transaction ID],
    [Departure Station],
    [Arrival Destination],
    [Date of Journey],
    [Price]
FROM
    Sheet1$
ORDER BY
    [Date of Journey];
```

	Transaction ID	Departure Station	Arrival Destination	Date of Journey	Price
1	52477111-4b67-4d87-9ab4	London Paddington	Reading	2024-01-01 00:00:00.000	13
2	2b402b45-0762-485e-a7a3	Liverpool Lime Street	London Euston	2024-01-01 00:00:00.000	76
3	4e1dc885-3d95-44ef-99fa	London Kings Cross	York	2024-01-01 00:00:00.000	35
4	1c74479d-85a4-4ba1-a607	London Euston	Oxford	2024-01-01 00:00:00.000	2
5	feb8dab-608b-46fa-b2b	Liverpool Lime Street	Manchester Piccadilly	2024-01-01 00:00:00.000	2
6	01d916f4-2911-41ec-a37d	London Euston	York	2024-01-01 00:00:00.000	37
7	a0cedba7-1923-459d-b046	London Paddington	Reading	2024-01-01 00:00:00.000	13
8	b3e5ca7d-e76c-49c2-b49f	York	Durham	2024-01-01 00:00:00.000	8
9	6ef397ac-d590-4356-9eaa	London Paddington	Reading	2024-01-01 00:00:00.000	8
10	2a7a6d75-566a-41aa-9468	Manchester Piccadilly	Liverpool Lime Street	2024-01-01 00:00:00.000	3
11	7ed95d45-e66f-49b2-9b5a	Manchester Piccadilly	Liverpool Lime Street	2024-01-01 00:00:00.000	3
12	2e05a2a6-88a9-40fb-bacc	Birmingham New Street	London St Pancras	2024-01-01 00:00:00.000	15
13	8a18d3b4-995e-49bf-93a3	London Paddington	Reading	2024-01-01 00:00:00.000	13
14	7493a611-342a-4b17-90dc	London Paddington	Reading	2024-01-01 00:00:00.000	13
15	054678ac-a976-4909-a26e	London St Pancras	Birmingham New Street	2024-01-01 00:00:00.000	8
16	6b62b452-c491-468d-b39c	London Kings Cross	York	2024-01-01 00:00:00.000	35
17	85e38992-6b6c-4569-914e	London Kings Cross	York	2024-01-01 00:00:00.000	35

Cypher Query:

```
MATCH (j:Journey)-[:STARTS_AT]->(src:Station), (j)-[:ENDS_AT]->(dest:Station)
RETURN j.id AS TransactionID, src.name AS DepartureStation, dest.name AS ArrivalStation, j.date AS
DateOfJourney, j.price AS Price
ORDER BY j.date;
```

This retrieves journeys from departure to arrival stations, along with their travel dates and price, and lists them in ascending order of journey date.

6. Late Arrivals (Actual Arrival > Scheduled Arrival)

SQL Query:

```
SELECT [Transaction ID], [Departure Station], [Arrival Destination], [Arrival Time], [Actual Arrival Time]
FROM Sheet1$
WHERE TRY_CAST([Actual Arrival Time] AS TIME) > TRY_CAST([Arrival Time] AS TIME);
```

This query identifies journeys where the actual arrival time is later than the scheduled arrival. It compares time values using TRY_CAST for safe type conversion.

```

---query 6 Late Arrivals (Actual Arrival > Scheduled Arrival)---
SELECT
  [Transaction ID],
  [Departure Station],
  [Arrival Destination],
  [Arrival Time],
  [Actual Arrival Time]
FROM
  Sheet1$
WHERE
  TRY_CAST([Actual Arrival Time] AS TIME) > TRY_CAST([Arrival Time] AS TIME);

```

Transaction ID	Departure Station	Arrival Destination	Arrival Time	Actual Arrival Time
b31b38f1-e3d4-4357-9b40	Manchester Piccadilly	Liverpool Lime Street	1899-12-30 02:00:00.000	1899-12-30 02:44:00.000
f948304-9147-4b3b-827d	London Paddington	Reading	1899-12-30 05:15:00.000	1899-12-30 05:52:00.000
886da3-49e7-4b7b-94ec	London Kings Cross	York	1899-12-30 08:05:00.000	1899-12-30 08:34:00.000
f284543-c0b-4e2d-8dce	London Kings Cross	York	1899-12-30 08:05:00.000	1899-12-30 08:34:00.000
24a3ae5d-77d3-474c-9c5d	London Kings Cross	York	1899-12-30 08:05:00.000	1899-12-30 08:34:00.000
4474018-3cd0-4530-5e4f	Manchester Piccadilly	Leeds	1899-12-30 09:45:00.000	1899-12-30 12:26:00.000
5e08481-4e5d-4e25-87d5	Liverpool Lime Street	London Euston	1899-12-30 10:15:00.000	1899-12-30 11:07:00.000
88a79c0b-c07b-4b44-907a	Liverpool Lime Street	London Euston	1899-12-30 10:15:00.000	1899-12-30 11:07:00.000
495296d-3e4d-4d2b-5e45	Liverpool Lime Street	London Euston	1899-12-30 10:15:00.000	1899-12-30 11:07:00.000
0e17a389-8e6f-492d-a625	Manchester Piccadilly	London Euston	1899-12-30 09:50:00.000	1899-12-30 10:47:00.000
60351e6f-4448-4685-8957	Liverpool Lime Street	London Euston	1899-12-30 10:15:00.000	1899-12-30 11:07:00.000
e06abe8-90b-4299-a610	Liverpool Lime Street	London Euston	1899-12-30 10:15:00.000	1899-12-30 11:07:00.000
705102ba-1739-45e4-a0a4	Manchester Piccadilly	London Euston	1899-12-30 09:50:00.000	1899-12-30 10:47:00.000
e046c97-0167-413c-9748	Liverpool Lime Street	London Euston	1899-12-30 10:15:00.000	1899-12-30 11:07:00.000
97173ab7-a0b4-4081-85c4	Manchester Piccadilly	Nottingham	1899-12-30 10:00:00.000	1899-12-30 10:41:00.000

Cypher Query:

MATCH (j:Journey)-[:STARTS_AT]->(src), (j)-[:ENDS_AT]->(dest)

WHERE j.actual_arrival_time > j.arrival_time

RETURN j.id AS TransactionID, src.name AS DepartureStation, dest.name AS ArrivalStation, j.arrival_time, j.actual_arrival_time;

In Neo4j, it checks if the actual_arrival_time on a journey node is greater than its arrival_time and returns such records with station details and timing.

```

1 MATCH (j:Journey)-[:STARTS_AT]->(src),
2   (j)-[:ENDS_AT]->(dest)
3 WHERE j.actual_arrival_time > j.arrival_time
4 RETURN
5   j.id AS TransactionID,
6   src.name AS DepartureStation,
7   dest.name AS ArrivalStation,
8   j.arrival_time,
9   j.actual_arrival_time;

```

TransactionID	DepartureStation	ArrivalStation	j.arrival_time	j.actual_arrival_time
"b0cdd1b0-f214-4197-be53"	"London Kings Cross"	"York"	"11:35:00"	"12/30/1899"
"febf8dab-f808-46fa-bf2b"	"Liverpool Lime Street"	"Manchester Piccadilly"	"0:30:00"	"12/30/1899"
"85e38992-6c6c-4569-914e"	"London Kings Cross"	"York"	"10:50:00"	"12/30/1899"
"a478f358-044d-4000-b70e"	"Liverpool Lime Street"	"Manchester Piccadilly"	"11:00:00"	"12/30/1899"
"d4ea6940-7228-41ef-9eae"	"London Kings Cross"	"York"	"11:20:00"	"12/30/1899"
"a3a42b1b-bf7f-43d9-b63c"	"London Euston"	"Birmingham New Street"	"0:50:00"	"12/30/1899"
"382d60f9-9fe0-407a-07aa"	"Liverpool Lime Street"	"London Euston"	"10:15:00"	"12/30/1899"

Started streaming 3501 records.

7. GetJourneyDetails_SQL

SQL Query:

SELECT [Transaction ID], [Departure Station], [Arrival Destination], [Date Of Journey], [Journey Status] FROM Sheet1\$ ORDER BY [Date Of Journey];

Retrieves journey data from an Excel sheet where all journey details are stored in a flat table format

-- query 7 GetJourneyDetails_SQL----

SELECT

[Transaction ID],

[Departure Station],

[Arrival Destination],

[Date Of Journey],

[Journey Status]

FROM

Sheet1\$

ORDER BY

[Date Of Journey];

10 %

ResultsMessages

	Transaction ID	Departure Station	Arrival Destination	Date Of Journey	Journey Status
1	b2471f11-4e7-4c87-8ab4	London Paddington	Reading	2024-01-01 00:00:00.000	On Time
2	2be00b45-0762-485e-a7a3	Liverpool Lime Street	London Euston	2024-01-01 00:00:00.000	On Time
3	4e1dcd88-3d95-44ef-99fa	London Kings Cross	York	2024-01-01 00:00:00.000	On Time
4	1c74479d-85a4-4ba1-a607	London Euston	Oxford	2024-01-01 00:00:00.000	On Time
5	fab86a8b-8038-469a-9d2b	Liverpool Lime Street	Manchester Piccadilly	2024-01-01 00:00:00.000	On Time
6	01a89164-4291-41ee-a37d	London Euston	York	2024-01-01 00:00:00.000	Delayed
7	a8cedba7-1923-459d-b046	London Paddington	Reading	2024-01-01 00:00:00.000	On Time
8	b3e5ca7d-e76c-4902-b49f	York	Durham	2024-01-01 00:00:00.000	On Time
9	6c637ac-d590-4356-9eaa	London Paddington	Reading	2024-01-01 00:00:00.000	On Time
10	2e7add75-566a-41aa-9468	Manchester Piccadilly	Liverpool Lime Street	2024-01-01 00:00:00.000	On Time
11	7ed9b545-eb6f-49b2-9b5a	Manchester Piccadilly	Liverpool Lime Street	2024-01-01 00:00:00.000	On Time
12	2e05e2a6-88a8-40b-bacc	Birmingham New Street	London St Pancras	2024-01-01 00:00:00.000	On Time
13	8a18d3b4-995e-49b6-93a3	London Paddington	Reading	2024-01-01 00:00:00.000	On Time
14	7493a611-342a-4b17-90dc	London Paddington	Reading	2024-01-01 00:00:00.000	On Time
15	054676ac-a97b-4909-a26e	London St Pancras	Birmingham New Street	2024-01-01 00:00:00.000	On Time

Cypher Query:

MATCH (j:Journey)-[:STARTS_AT]->(src:Station), (j)-[:ENDS_AT]->(dest:Station) RETURN j.id AS TransactionID, src.name AS DepartureStation, dest.name AS ArrivalStation, j.date AS DateOfJourney, j.status AS JourneyStatus ORDER BY j.date;

Retrieves journey data by traversing relationships between Journey and Station nodes in a graph database.

1 MATCH (j:Journey)-[:STARTS_AT]->(src:Station),
2 (j)-[:ENDS_AT]->(dest:Station)
3 RETURN
4 j.id AS TransactionID,
5 src.name AS DepartureStation,
6 dest.name AS ArrivalStation,
7 j.date AS DateOfJourney,
8 j.status AS JourneyStatus
9 ORDER BY j.date;

TableRAW

TransactionID	DepartureStation	ArrivalStation	DateOfJourney	JourneyStatus
"842d993c-b820-42dc-ad4f"	"Manchester Piccadilly"	"London Paddington"	"1/1/2024"	"On Time"
"da8a6ba8-b3dc-4677-b176"	"London Paddington"	"Liverpool Lime Street"	"1/1/2024"	"On Time"
"2e7add75-566a-41aa-9468"	"Manchester Piccadilly"	"Liverpool Lime Street"	"1/1/2024"	"On Time"
"7ed9b545-eb6f-49b2-9b5a"	"Manchester Piccadilly"	"Liverpool Lime Street"	"1/1/2024"	"On Time"
"ea7108cf-382b-4794-aa8c"	"Manchester Piccadilly"	"Liverpool Lime Street"	"1/1/2024"	"On Time"
"5f7dd36c-6a06-49bc-9ff6"	"Manchester Piccadilly"	"Liverpool Lime Street"	"1/1/2024"	"On Time"
"1d704d30-6e8b-4b7f6-0d4e-1a3a"	"Manchester Piccadilly"	"Liverpool Lime Street"	"1/1/2024"	"On Time"

Fetch limit hit at 5000 records. Started streaming after 752 ms and completed after 1,103 ms.

Summary of Observations

Criteria	SQL (Relational DB)	CQL (Graph DB - Neo4j)
Schema Design	Tabular, fixed schema	Node-relationship model, flexible
Query Complexity	Increases with more joins	Simplified with pattern matching
Relationship Handling	Indirect (via foreign keys)	Direct and intuitive
Performance	Slower with complex joins	Faster for deeply connected data
Use Case Fit	Ideal for tabular data and reporting	Best for relationships and graph analytics

Comparison

1. Journey Count per Departure Station

- **SQL:** Performed better due to efficient grouping and indexing.
- **Neo4j:** Worked correctly but was slightly slower due to traversal and pattern matching.

2. Journeys with Price Greater than 50

- **SQL:** Performed well; simple numeric filtering was quick.
- **Neo4j:** Performed similarly; property filtering was straightforward and fast.
- **Performance:** Comparable in both.

3. First Class Tickets

- **SQL:** Efficient with basic filtering.
- **Neo4j:** Also, efficient and more intuitive to read.
- **Performance:** Comparable in both.

4. Stations with More Than 5 Departing Journeys

- **SQL:** Performed better; aggregation with GROUP BY and HAVING was optimized.
- **Neo4j:** Slightly slower due to additional pattern matching and the use of WITH.

5. Journeys Ordered by Date

- **SQL:** Performed better; indexing made sorting by date fast.
- **Neo4j:** Functional but less efficient due to traversal involved.

6. Late Arrivals (Actual Arrival > Scheduled Arrival)

- **SQL:** Worked well but needed data type conversion, adding some complexity.
- **Neo4j:** Performed better as date/time comparisons were more direct and readable.

7. Get Journey Details (Basic Info with Status and Date)

- **SQL:** Performed better with flat, direct column retrieval.
- **Neo4j:** Required extra relationship traversal, making it slightly slower.

Overall Observation:

- **SQL performed better** in: Journey Count, Stations with >5 Departures, Journeys Ordered by Date, and Get Journey Details.
- **Neo4j performed better** in: Late Arrivals.
- **Both performed equally well** in: Price > 50- and First-Class Tickets.

6. Conclusion

This project has been able to present an all-inclusive Proof of Concept (PoC) on the construction of data warehouse solution that would integrate into Railway Ticketing and Journey Analytics. Through conversion of raw transactional data to stored star schema and an end-to-end ETL process through SSIS, historical and in-time business intelligence across the departments is now aided efficiently through the data warehouse. The design supports multidimensional analysis with the help of properly designed fact and dimension tables to allow stakeholders, such as operations managers, finance departments, customer support and marketing analysts, to base their decisions on evidence. The major insights on the business that the railway organization can get thanks to the warehouse, including journey punctuality, pattern of ticket sales, patterns of refund, and payment behaviours allow enhancing the services offered, enhancing customer experience, and increasing revenue. The solution is further enhanced by:

- **SSRS Reports** (tabular, matrix, parameterized, drill-down) enabling diverse report delivery.
- **Tableau Dashboards** offering visual, interactive insights to key stakeholders.
- **ETL Pipelines** that ensure data integrity, quality, and performance.
- **SQL vs Graph DB (Neo4j)** comparison, revealing context-specific advantages of each database type.

Its **scalable architecture** provides a good basis to future-proof the data warehouse by extending it with more advanced features, like machine learning driven forecasting, anomaly detection and operational dashboards in real-time. It can be considered a strategic transition to the culture of data and the digital turning of the railway industry.

7. References / Bibliography

- Microsoft. (n.d.). *SQL Server Integration Services (SSIS) Documentation*. Retrieved from: <https://learn.microsoft.com/en-us/sql/integration-services/>
- Tableau. (n.d.). *Data Visualization Best Practices*. Retrieved from: <https://www.tableau.com/>
- Microsoft. (n.d.). *SQL Server Reporting Services (SSRS)*. Retrieved from: <https://learn.microsoft.com/en-us/sql/reporting-services/>
- Neo4j. (n.d.). *Cypher Query Language Manual*. Retrieved from: <https://neo4j.com/docs/cypher-manual/>

8. Individual Contribution Report

Working on this data warehousing project provided a highly valuable and insightful learning experience. It allowed me to deepen my understanding of real-world data processing, integration, and analytics. My key contribution to this project was in developing and executing the **ETL process using SSIS**, designing the **data warehouse schema in SSMS**, and creating **reports using SSRS**.

To begin, I worked extensively on **SQL Server Management Studio (SSMS)** to create the dimensional model based on the star schema. This involved designing the fact and dimension tables, ensuring referential integrity through surrogate keys, and loading cleaned and validated data. I also defined appropriate data types and handled indexing to support optimized queries.

In **SQL Server Integration Services (SSIS)**, I developed the complete ETL workflow. I handled the data extraction from the Excel source, applied necessary transformations such as data standardization, and used multiple lookup transformations to map natural keys to surrogate keys in dimension tables. Ensuring **referential integrity and data consistency** throughout the pipeline was a major focus. I also verified the load success using SQL queries on the populated warehouse tables.

For **reporting**, I created multiple **SSRS reports**, including a tabular report to show detailed transaction data, a matrix report for revenue comparison by ticket class and payment method, and a parameterized report allowing users to dynamically explore ticket sales. These reports offer useful insights for various stakeholders and demonstrate how well-structured data supports strategic decision-making.

In addition to my contributions, I would like to acknowledge the efforts of my teammates:

- **Harsh** (myself) – Led the work on **SSMS, SSIS, and SSRS**.
- **Gunasekhar Vasireddy** played a crucial role in implementing the **graph database component using Neo4j**. He helped write Cypher queries and supported the comparison with relational SQL queries, which enriched our understanding of graph-based analytics.
- **Harsanbruno Maria Joseph Esuraj** was instrumental in creating **visual dashboards using Tableau**. He also contributed to **SSRS** by supporting the design of the drill-down report and helped validate the accuracy of visuals generated from our data warehouse.

This project helped me improve my technical skills, enhance team collaboration, and gain confidence in working with BI tools and analytical thinking. It reinforced the importance of data quality, clean architecture, and collaboration in building meaningful data-driven solutions.

Appendix A: SQL Code – Data Warehouse Schema Creation

```
DROP DATABASE IF EXISTS Railway_DW_;

CREATE DATABASE Railway_DW_;

GO

-- Use the newly created database

USE Railway_DW_;

GO

-- Create Date Dimension Table

CREATE TABLE DW_Dim_Date1_ (

    DateKey INT IDENTITY(1,1) PRIMARY KEY,

    FullDate DATE NOT NULL,

    Year INT NOT NULL,

    Month INT NOT NULL,

    Day INT NOT NULL,
```

```

    DayOfWeek VARCHAR(10) NOT NULL
);
-- Create Station Dimension Table
CREATE TABLE DW_Dim_Station1_ (
    StationKey INT IDENTITY(1,1) PRIMARY KEY,
    StationCode VARCHAR(10) NOT NULL,
    StationName NVARCHAR(255) NOT NULL,
    Region VARCHAR(50),
    Zone VARCHAR(20)
);
-- Create Ticket Dimension Table
CREATE TABLE DW_Dim_Ticket1_ (
    TicketKey INT IDENTITY(1,1) PRIMARY KEY,
    TicketType VARCHAR(50) NOT NULL,
    TicketClass VARCHAR(50) NOT NULL,
    Railcard VARCHAR(100) NOT NULL,
    DiscountPercentage FLOAT
);
-- Modify columns in Ticket Dimension Table
ALTER TABLE DW_Dim_Ticket1_
ALTER COLUMN Railcard VARCHAR(100) NULL;
ALTER TABLE DW_Dim_Ticket1_
ALTER COLUMN TicketType VARCHAR(100);
-- Create Payment Dimension Table
CREATE TABLE DW_Dim_Payment1_ (
    PaymentKey INT IDENTITY(1,1) PRIMARY KEY,
    PurchaseType NVARCHAR(50) NOT NULL,
    PaymentMethod NVARCHAR(50) NOT NULL
);
-- Create Fact Table for Journey Data
CREATE TABLE DW_Fact_Journey1_ (

```

```

FactID INT IDENTITY(1,1) PRIMARY KEY,
PurchaseDateKey INT NOT NULL,
JourneyDateKey INT NOT NULL,
DepartureStationKey INT NOT NULL,
ArrivalStationKey INT NOT NULL,
TicketKey INT NOT NULL,
PaymentKey INT NOT NULL,
Price FLOAT NOT NULL,
DepartureTime TIME NOT NULL,
ArrivalTime TIME NOT NULL,
ActualArrivalTime TIME,
JourneyStatus VARCHAR(50) NOT NULL,
RefundRequest VARCHAR(10) NOT NULL,
FOREIGN KEY (PurchaseDateKey) REFERENCES DW_Dim_Date1_(DateKey),
FOREIGN KEY (JourneyDateKey) REFERENCES DW_Dim_Date1_(DateKey),
FOREIGN KEY (DepartureStationKey) REFERENCES DW_Dim_Station1_(StationKey),
FOREIGN KEY (ArrivalStationKey) REFERENCES DW_Dim_Station1_(StationKey),
FOREIGN KEY (TicketKey) REFERENCES DW_Dim_Ticket1_(TicketKey),
FOREIGN KEY (PaymentKey) REFERENCES DW_Dim_Payment1_(PaymentKey)
);

```

```

-- Sample Queries to View Populated Tables

```

```

SELECT * FROM DW_Dim_Payment1_;

```

```

SELECT * FROM DW_Dim_Station1_;

```

```

SELECT * FROM DW_Dim_Ticket1_;

```

```

SELECT * FROM DW_Dim_Date1_;

```

```

-- Modify FullDate to store full datetime if required

```

```

ALTER TABLE DW_Dim_Date1_

```

```

ALTER COLUMN FullDate DATETIME;

```